THE NATIONAL UNIVERSITY OF LESOTHO

Department of Mathematics and Computer Science

CS3520 - Computer Organisation and Architecture I

Tutorial 2

28th October 2025

1. Consider the following instruction:

   Instruction: and rd, rs1, rs2

   Interpretation: Reg[rd] = Reg[rs1] AND Reg[rs2] 4.1.1

   (a) What are the values of control signals generated by the control in for this instruction?

   (b) Which resources (blocks) perform a useful function for this instruction?

   (c) Which resources (blocks) produce no output for this instruction?

   (d) Which resources produce output that is not used?

2. When silicon chips are fabricated, defects in materials (e.g., silicon) and manufacturing errors can result in defective circuits. A very common defect is for one signal wire to get "broken" and always register a logical 0. This is often called a "stuck-at-0" fault.

   (a) Which instructions fail to operate correctly if the MemToReg wire is stuck at 0?

   (b) Which instructions fail to operate correctly if the ALUSrc wire is stuck at 0?

3. In this exercise, we examine in detail how an instruction is executed in a single-cycle datapath. Problems in this exercise refer to a clock cycle in which the processor fetches the following instruction word: **0x00c6ba23**.

   (a) What are the values of the ALU control unit's inputs for this instruction?

   (b) What is the new PC address after this instruction is executed? Highlight the path through which this value is determined.

   (c) For each mux, show the values of its inputs and outputs during the execution of this instruction. List values that are register outputs at Reg [xn].

   (d) What are the input values for the ALU and the two add units?

   (e) What are the values of all inputs for the registers unit?

4. Problems in this exercise assume that the logic blocks used to implement a processor's datapath have the following latencies:

   "Register read" is the time needed after the rising clock edge for the new register value to appear on the output. This value applies to the PC only. "Register setup" is the amount of time a register's data input must be stable before the rising edge of the clock. This value applies to both the PC and Register File.

   (a) What is the latency of an R-type instruction (i.e., how long must the clock period be to ensure that this instruction works correctly)?

   (b) What is the latency of lw? (Check your answer carefully. Many students place extra muxes on the critical path.)

(c) What is the latency of sw? (Check your answer carefully. Many students place extra muxes on the critical path.)

(d) What is the latency of beq?

(e) What is the latency of an arithmetic, logical, or shift I-type (non-load) instruction?

(f) What is the minimum clock period for this CPU?

5. Consider the addition of a multiplier to the CPU shown in *Slide 62* of the lecture slides. This addition will add 300 ps to the latency of the ALU, but will reduce the number of instructions by 5% (because there will no longer be a need to emulate the multiply instruction).

   (a) What is the clock cycle time with and without this improvement?

   (b) What is the speedup achieved by adding this improvement?

   (c) What is the slowest the new ALU can be and still result in improved performance?

6. Examine the difficulty of adding a proposed *lwi.d rd, rs1, rs2* ("Load With Increment") instruction to RISC-V.

   Interpretation: Reg[rd]=Mem[Reg[rs1]+Reg[rs2]]

   (a) Which new functional blocks (if any) do we need for this instruction?

   (b) Which existing functional blocks (if any) require modification?

   (c) Which new data paths (if any) do we need for this instruction?

   (d) What new signals do we need (if any) from the control unit to support this instruction?

7. Examine the difficulty of adding a proposed swap rs1, rs2 instruction to RISC-V. Interpretation: Reg[rs2]=Reg[rs1]; Reg[rs1]=Reg[rs2]

   (a) Which new functional blocks (if any) do we need for this instruction?

   (b) Which existing functional blocks (if any) require modification?

   (c) What new data paths do we need (if any) to support this instruction?

   (d) What new signals do we need (if any) from the control unit to support this instruction?

   (e) Modify the datapath and control in *Slide 62* of the lecture slides to demonstrate an implementation of this new instruction.

8. For which instructions (if any) is the *Imm Gen* block on the critical path?

9. In this exercise, we examine how pipelining affects the clock cycle time of the processor. Problems in this exercise assume that individual stages of the datapath have the following latencies: Also, assume that instructions executed by the processor are broken down as follows:

   (a) What is the clock cycle time in a pipelined and non-pipelined processor?

   (b) What is the total latency of an lw instruction in a pipelined and non-pipelined processor?

   (c) If we can split one stage of the pipelined datapath into two new stages, each with half the latency of the original stage, which stage would you split and what is the new clock cycle time of the processor?

   (d) Assuming there are no stalls or hazards, what is the utilization of the data memory?

   (e) Assuming there are no stalls or hazards, what is the utilization of the write-register port of the "Registers" unit?

10. What is the minimum number of cycles needed to completely execute n instructions on a CPU with a k stage pipeline? Justify your formula.

11. Assume that x11 is initialized to 11 and x12 is initialized to 22. Suppose you executed the code below on a version of the pipeline that does not handle data hazards (i.e., the programmer is responsible for addressing data hazards by inserting NOP instructions where necessary). What would the final values of registers x13 and x14 be?

```
addi x11, x12, 5
addi x13, x11, x12
addi x14, x11, 15
```

12. Add NOP instructions to the code below so that it will run correctly on a pipeline that does not handle data hazards.

```
addi x11, x12, 5
addi x13, x11, x12
addi x14, x11, 15
addi x15, x13, 12
```

13. Consider the fragment of RISC-V assembly below:

```
sw x29, 12(x16)
lw x29, 8(x16)
sub x17, x15, x14
beqz x17, label
add x15, x11, x14
sub x15, x30, x14
```

Suppose we modify the pipeline so that it has only one memory (that handles both instructions and data). In this case, there will be a structural hazard every time a program needs to fetch an instruction during the same cycle in which another instruction accesses data.

(a) Draw a pipeline diagram to show were the code above will stall.

(b) In general, is it possible to reduce the number of stalls/NOPs resulting from this structural hazard by reordering code?

(c) Must this structural hazard be handled in hardware? We have seen that data hazards can be eliminated by adding NOPs to the code. Can you do the same with this structural hazard? If so, explain how. If not, explain why not.

14. Consider the following loop:

```
LOOP: lw x10, 0(x13)
    lw x11, 8(x13)
    add x12, x10, x11
    addi x13, x13, 16
    bnez x12, LOOP
```

Assume that perfect branch prediction is used (no stalls due to control hazards), that there are no delay slots, that the pipeline has full forwarding support, and that branches are resolved in the EX (as opposed to the ID) stage.

(a) Show a pipeline execution diagram for the first two iterations of this loop.

(b) Mark pipeline stages that do not perform useful work. How often while the pipeline is full do we have a cycle in which all five pipeline stages are doing useful work? (Begin with the cycle during which the addi is in the IF stage. End with the cycle during which the bnez is in the IF stage.)

15. Problems in this exercise refer to the following sequence of instructions, and assume that it is executed on a five-stage pipelined datapath:

```
add x15, x12, x11
lw x13, 8(x15)
lw x12, 0(x2)
or x13, x15, x13
sw x13, 0(x15)
```

(a) If there is no forwarding or hazard detection, insert NOPs to ensure correct execution.

(b) Now, change and/or rearrange the code to minimize the number of NOPs needed. You can assume register x17 can be used to hold temporary values in your modified code.

(c) If the processor has forwarding, but we forgot to implement the hazard detection unit, what happens when the original code executes?

(d) If there is forwarding, for the first seven cycles during the execution of this code, specify which signals are asserted in each cycle by hazard detection and forwarding units in *Slide 134*.

(e) If there is no forwarding, what new input and output signals do we need for the hazard detection unit in *Slide 134*? Using this instruction sequence as an example, explain why each signal is needed.

16. This exercise examines the accuracy of various branch predictors for the following repeating pattern (e.g., in a loop) of branch outcomes: T, NT, T, T, NT.

(a) What is the accuracy of always-taken and always-not-taken predictors for this sequence of branch outcomes?

(b) What is the accuracy of the 2-bit predictor for the first four branches in this pattern, assuming that the predictor starts off in the bottom left state in *Slide 145* (predict not taken)?

(c) What is the accuracy of the 2-bit predictor if this pattern is repeated forever?

(d) Design a predictor that would achieve a perfect accuracy if this pattern is repeated forever. You predictor should be a sequential circuit with one output that provides a prediction (1 for taken, 0 for not taken) and no inputs other than the clock and the control signal that indicates that the instruction is a conditional branch.

(e) What is the accuracy of your predictor from (e) if it is given a repeating pattern that is the exact opposite of this one?

(f) Repeat (d), but now your predictor should be able to eventually (after a warm-up period during which it can make wrong predictions) start perfectly predicting both this pattern and its opposite. Your predictor should have an input that tells it what the real outcome was. Hint: this input lets your predictor determine which of the two repeating patterns it is given.

17. This exercise explores how exception handling affects pipeline design. The first three problems in this exercise refer to the following two instructions:

instruction1	**instruction2**
beqz x11, LABEL	lw x11, 0(x12)

(a) Which exceptions can each of these instructions trigger? For each of these exceptions, specify the pipeline stage in which it is detected.

(b) If there is a separate handler address for each exception, show how the pipeline organization must be changed to be able to handle this exception. You can assume that the addresses of these handlers are known when the processor is designed.

(c) If the second instruction is fetched immediately after the first instruction, describe what happens in the pipeline when the first instruction causes the first exception you listed in (a). Show the pipeline execution diagram from the time the first instruction is fetched until the time the first instruction of the exception handler is completed.

(d) In vectored exception handling, the table of exception handler addresses is in data memory at a known (fixed) address. Change the pipeline to implement this exception handling mechanism. Repeat (c) using this modified pipeline and vectored exception handling.